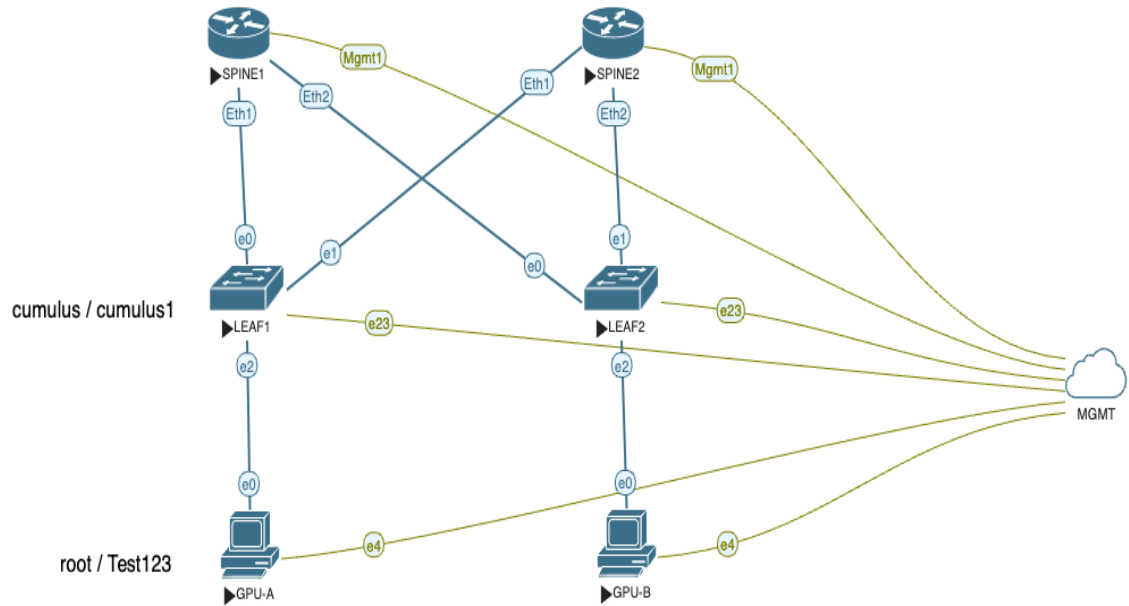


QoS / IPv6 Practice Workbook

Step-by-step additive exercises for QoS, IPv6, ECN, WRED, strict priority, PFC boundaries, and RDMA regression on the current working EVE topology.



Current validated working topology used by this workbook. Management cloud wiring remains external and environment-specific.

QoS / IPv6 Practice Workbook

EVE-NG RoCEv2 / MPLS-LDP / EVPN-VXLAN Working Topology

Author

- Mohammad Tahir
- tahiritian@gmail.com

1. Purpose

This workbook adds new practice modules on top of the already validated working lab:

- QoS classification
- DSCP marking
- WRED
- ECN
- strict priority queueing concepts
- IPv6 over the current overlay
- soft-RoCE traffic validation after changes

The key design rule for this workbook is:

- do not break the current working underlay
- do not break the current working overlay
- keep OSPF, MPLS/LDP, EVPN, VXLAN, and RXE intact

That means all practice steps are either:

- additive
- reversible
- intentionally scoped to host-facing or host-side software features unless explicitly noted

2. Important Reality Check

This current EVE lab is excellent for control-plane learning and end-to-end traffic testing, but not every data-center QoS feature behaves the same in a virtual lab as on real ASIC-based hardware.

2.1 What This Workbook Can Validate Well

- IPv6 reachability over the current EVPN/VXLAN overlay
- DSCP marking and preservation
- Linux software queueing examples
- WRED/RED with ECN in Linux software
- strict-priority style queue ordering in Linux software
- RDMA reachability after non-disruptive changes

2.2 What This Workbook Only Approximates

- WRED behavior on a Linux software qdisc instead of switch ASIC WRED
- ECN marking in Linux software instead of merchant-silicon queue hardware
- strict priority queueing in Linux qdisc instead of hardware egress schedulers

2.3 What This Workbook Does Not Truly Validate

- real link-layer PFC pause frame generation and reaction on hardware switch ASICs
- real NIC hardware RoCE congestion control behavior
- hardware ECN/DCQCN pipeline validation

For PFC in this topology, the workbook uses:

- conceptual steps
- observation boundaries
- explicit “do not assume hardware equivalence” guidance

3. Current Working Baseline

Before starting any new practice module, confirm the fabric is healthy.

3.1 Minimum Baseline

- OSPF full between leaves and spines
- MPLS/LDP operational
- EVPN established
- VTEP reachability working
- VXLAN host reachability working

- RXE device present on both GPU hosts
- `ib\_write\_bw` able to connect

Run:

```
cd /Users/mtahir/eveng_rocev2_mpls_vxlan_lab
cat VALIDATION_CHECKLIST.md
```

Then run the commands from:

- `VALIDATION\_CHECKLIST.md`
- `OVERLAY\_VALIDATION.md`
- `ROCE\_VALIDATION.md`

Do not continue until baseline is clean.

4. Rollback Strategy

Every step in this workbook should be reversible.

4.1 Host-Side Rollback

Most QoS experiments in this workbook are intentionally host-side or host-facing.

To clear Linux `tc` qdisc changes:

```
tc qdisc del dev ens3 root 2>/dev/null
tc qdisc del dev ens3 ingress 2>/dev/null
```

To clear IPv6 test addresses:

```
ip -6 addr flush dev ens3 scope global
```

To clear temporary DSCP iptables rules:

```
iptables -t mangle -F
ip6tables -t mangle -F
```

4.2 Leaf-Side Rollback

If you apply any optional Linux `tc` policy on leaf host-facing ports:

```
tc qdisc del dev swp2 root 2>/dev/null
tc qdisc del dev swp2 ingress 2>/dev/null
```

Do not modify:

- OSPF
- MPLS/LDP
- BGP EVPN
- loopbacks
- VNI or bridge configuration

unless you are deliberately rebuilding the lab, which this workbook does not require.

5. Workbook Order

Use this order:

1. baseline validation
2. IPv6 overlay validation
3. DSCP / QoS marking practice
4. strict-priority queue practice
5. RED/WRED-style queue practice
6. ECN marking practice
7. PFC conceptual review and limits
8. RoCE/RDMA regression validation

Each step should be completed before moving to the next.

6. Step 1: Baseline Snapshot

This step does not change anything. It creates a known-good checkpoint.

6.1 On LEAF-1

```
ip -br addr
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show mpls ldp neighbor'
vtysh -c 'show bgp l2vpn evpn summary'
bridge link
ip -d link show vni10
```

Healthy:

- 2 OSPF neighbors
- 2 LDP neighbors

- 2 EVPN peers
- `swp2` and `vni10` in `br0`

6.2 On LEAF-2

```
ip -br addr
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show mpls ldp neighbor'
vtysh -c 'show bgp l2vpn evpn summary'
bridge link
ip -d link show vni10
```

6.3 On GPU-A

```
ip -br addr
rdma link show
ibv_devices
ping -c 3 172.16.10.12
```

6.4 On GPU-B

```
ip -br addr
rdma link show
ibv_devices
ping -c 3 172.16.10.11
```

6.5 On SPINE1 and SPINE2

```
show ip ospf neighbor
show mpls ldp neighbor
show bgp evpn summary
```

If all of that is healthy, continue.

7. Step 2: IPv6 Overlay Practice

This is the safest new feature to add first because it does not disturb the existing IPv4 control plane.

7.1 Goal

Extend IPv6 host communication across the already working VXLAN bridge domain between:

- `GPU-A ens3`
- `GPU-B ens3`

7.2 Configure GPU-A

```
ip -6 addr add 2001:db8:10::11/64 dev ens3
ip -6 addr show dev ens3
```

Expected:

- `2001:db8:10::11/64` appears on `ens3`

7.3 Configure GPU-B

```
ip -6 addr add 2001:db8:10::12/64 dev ens3
ip -6 addr show dev ens3
```

Expected:

- `2001:db8:10::12/64` appears on `ens3`

7.4 Validate IPv6 Host Reachability

On `GPU-A`:

```
ping6 -c 3 2001:db8:10::12
ip -6 neigh
```

On `GPU-B`:

```
ping6 -c 3 2001:db8:10::11
ip -6 neigh
```

Healthy:

- `ping6` succeeds both ways
- neighbor discovery resolves

7.5 What This Proves

Because the overlay is L2 extension, IPv6 rides across the same bridge/VXLAN domain without needing new IPv6 routing on the fabric.

This step proves:

- the overlay is protocol-agnostic at L2
- the VNI is carrying both IPv4 and IPv6 host traffic

7.6 Rollback

```
ip -6 addr del 2001:db8:10::11/64 dev ens3
ip -6 addr del 2001:db8:10::12/64 dev ens3
```

8. Step 3: DSCP / QoS Marking Practice

This step practices traffic marking without changing the control plane.

8.1 Goal

Create distinct classes of host-generated traffic:

- RoCE-like traffic: DSCP 26
- control-like traffic: DSCP 48
- best effort traffic: DSCP 0

8.2 Fast Method Using `ping`

On Linux, `ping -Q` can set the IPv4 TOS byte.

Useful values:

- DSCP 26 shifted into TOS: `0x68`
- DSCP 48 shifted into TOS: `0xc0`
- DSCP 0: `0x00`

8.3 Validate Best-Effort

On `GPU-B`, observe:

```
tcpdump -ni ens3 -vv icmp
```

On `GPU-A`, generate:

```
ping -Q 0x00 -c 3 172.16.10.12
```

Expected:

- packets arrive with default TOS / DSCP behavior

8.4 Validate RoCE-Like DSCP 26

On `GPU-B`, keep `tcpdump` running, then on `GPU-A`:

```
ping -Q 0x68 -c 3 172.16.10.12
```

Expected:

- packet capture shows TOS corresponding to DSCP 26

8.5 Validate Control-Like DSCP 48

On `GPU-A`:

```
ping -Q 0xc0 -c 3 172.16.10.12
```

Expected:

- packet capture shows high-priority DSCP marking

8.6 Optional Persistent Marking With iptables

On `GPU-A`, mark traffic to `GPU-B` as DSCP 26:

```
iptables -t mangle -A OUTPUT -d 172.16.10.12 -p icmp -j DSCP --set-dscp 26
iptables -t mangle -L -n -v
```

Validate with:

```
ping -c 3 172.16.10.12
```

Expected:

- capture on `GPU-B` shows DSCP 26

8.7 Rollback

```
iptables -t mangle -F
```

9. Step 4: Strict Priority Practice

This is a software approximation of strict priority.

9.1 Goal

Create separate traffic bands and place high-priority traffic into the preferred band.

9.2 Why Host-Side Practice Is Safer

Changing Linux qdiscs on GPU hosts is less risky than modifying leaf control-plane or uplink behavior.

9.3 Apply a Priority Qdisc on GPU-A

```
tc qdisc replace dev ens3 root handle 1: prio bands 4
tc qdisc show dev ens3
```

Expected:

- `prio` qdisc is installed

9.4 Add DSCP-Based Filters

Map traffic classes:

- DSCP 48 -> highest priority band
- DSCP 26 -> next band
- everything else -> default band

```
tc filter add dev ens3 protocol ip parent 1: prio 1 u32 match ip tos 0xc0 0xfc flowid 1:1
tc filter add dev ens3 protocol ip parent 1: prio 2 u32 match ip tos 0x68 0xfc flowid 1:2
tc filter show dev ens3 parent 1:
```

9.5 Generate Traffic

Best effort:

```
ping -Q 0x00 -c 5 172.16.10.12
```

Control-like:

```
ping -Q 0xc0 -c 5 172.16.10.12
```

RoCE-like:

```
ping -Q 0x68 -c 5 172.16.10.12
```

9.6 Validate Queue Usage

```
tc -s qdisc show dev ens3
tc -s filter show dev ens3 parent 1:
```

Healthy:

- counters increment under the expected filters
- traffic classes are being separated

9.7 What This Proves

This does not prove ASIC strict-priority scheduling, but it does prove:

- traffic can be classified by DSCP
- different traffic classes can be assigned to different queues

9.8 Rollback

```
tc qdisc del dev ens3 root
```

10. Step 5: WRED-Style Practice

True hardware WRED is ASIC-based. In this virtual lab, use Linux RED as the safe approximation.

10.1 Goal

Practice queue drop / early-mark behavior under congestion using software RED.

10.2 Where To Apply It

Apply it on `GPU-A ens3` or `GPU-B ens3` first. That keeps the fabric control plane untouched.

10.3 Apply RED on GPU-A

```
tc qdisc replace dev ens3 root red limit 400000 min 30000 max 60000 avpkt 1500 burst 20 bandwidth 1000mbit probabilistic
tc qdisc show dev ens3
```

Expected:

- `red` qdisc appears on `ens3`

10.4 Validate Statistics

Generate traffic:

```
ping -f -c 5000 172.16.10.12
```

Then inspect:

```
tc -s qdisc show dev ens3
```

Healthy:

- qdisc statistics increment
- depending on offered load, drop counters may increment

10.5 What To Look For

- packet counters rising
- overlimits or drops if traffic exceeds the threshold model

10.6 Rollback

```
tc qdisc del dev ens3 root
```

11. Step 6: ECN Practice

This step builds directly on the RED concept by enabling ECN marking instead of pure early drop.

11.1 Goal

Observe ECN-capable queue handling in software.

11.2 Apply RED With ECN

On `GPU-A`:

```
tc qdisc replace dev ens3 root red limit 400000 min 30000 max 60000 avpkt 1500 burst 20 ecn bandwidth 1000mbit prob  
tc qdisc show dev ens3
```

Expected:

- `red` qdisc installed with `ecn`

11.3 Generate ECN-Capable Traffic

Using `ping` will not fully emulate TCP ECN behavior, so treat this as a queue configuration exercise. If `iperf3` exists in your host images, use it. If it does not, continue with the queue-statistics practice only.

Optional check:

```
which iperf3
```

If present:

On `GPU-B`:

```
iperf3 -s
```

On `GPU-A`:

```
iperf3 -c 172.16.10.12 -t 20
```

Then inspect:

```
tc -s qdisc show dev ens3
```

11.4 What This Proves

This proves:

- a software queue can be configured to perform ECN-aware early marking behavior

It does not prove:

- switch ASIC ECN threshold behavior
- DCQCN reaction on real RoCE NICs

11.5 Rollback

```
tc qdisc del dev ens3 root
```

12. Step 7: Optional Host-Facing Leaf QoS Practice

Use this only after host-side tests are comfortable.

12.1 Goal

Apply Linux software queueing to the leaf host-facing access ports:

- `LEAF-1 swp2`
- `LEAF-2 swp2`

This keeps the underlay links untouched.

12.2 Apply RED on LEAF-1 Access Port

```
tc qdisc replace dev swp2 root red limit 400000 min 30000 max 60000 avpkt 1500 burst 20 ecn bandwidth 1000mbit prob  
tc qdisc show dev swp2
```

12.3 Apply RED on LEAF-2 Access Port

```
tc qdisc replace dev swp2 root red limit 400000 min 30000 max 60000 avpkt 1500 burst 20 ecn bandwidth 1000mbit prob  
tc qdisc show dev swp2
```

12.4 Validate

From `GPU-A`:

```
ping -f -c 5000 172.16.10.12
```

From each leaf:

```
tc -s qdisc show dev swp2
```

12.5 Rollback

On both leaves:

```
tc qdisc del dev swp2 root
```

13. Step 8: PFC Practice Boundary

This is the most important honesty section in the workbook.

13.1 What PFC Really Requires

True PFC validation normally requires:

- hardware NIC pause behavior
- switch ASIC priority-flow control support
- DCB/PFC negotiation or explicit hardware config
- pause counters and queue telemetry

13.2 What The Current EVE Lab Cannot Prove

The current virtual topology cannot truly validate:

- actual pause frame generation
- real per-priority pause reception
- ASIC queue halt/resume semantics

13.3 What You Can Practice Here

You can still use this lab to practice the design logic around PFC:

- keep loss-sensitive traffic isolated to a specific class
- do not apply pause behavior to broad traffic domains
- combine congestion signaling concepts with endpoint rate-control thinking

13.4 What To Record In Workbook Notes

When you document PFC in this environment, note:

- PFC is conceptual in this EVE build
- ECN/WRED can be approximated in software
- hardware pause validation needs real ASIC/NIC hardware

14. Step 9: Regression Test The Working Fabric

After any QoS experiment, re-run the critical checks.

14.1 Leaves

```
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show mpls ldp neighbor'
vtysh -c 'show bgp l2vpn evpn summary'
bridge link
```

Healthy:

- OSPF still full
- LDP still operational
- EVPN still established
- `vni10` still in `br0`

14.2 Hosts

```
ping -c 3 172.16.10.12
ping6 -c 3 2001:db8:10::12
rdma link show
ibv_devices
```

On `GPU-B`:

```
ping -c 3 172.16.10.11
ping6 -c 3 2001:db8:10::11
rdma link show
ibv_devices
```

14.3 Spine Checks

```
show ip ospf neighbor
show mpls ldp neighbor
show bgp evpn summary
```

15. Step 10: RDMA Regression Validation

After completing QoS exercises, confirm the RDMA path still works.

15.1 Server Side

On `GPU-B`:

```
ib_write_bw
```

Expected:

- waits for client
- prints connection details after client connects

15.2 Client Side

On `GPU-A`:

```
ib_write_bw 172.16.10.12
```

Expected:

- client connects
- bandwidth test starts

15.3 What Success Means

If `ib\_write\_bw` still connects after your QoS exercises, then:

- the overlay is still intact
- RXE is still intact
- the host path is still intact

16. Suggested Notes Template

Use this simple note format while practicing:

Module

- IPv6
- DSCP / QoS
- strict priority
- WRED
- ECN

- PFC concept
- RDMA regression

Change Applied

- exact command used
- which node it was applied on

Validation

- exact command used
- expected result
- actual result

Rollback

- exact rollback command

17. Recommended Safe Practice Path

If you want the least risk of breaking the working lab:

1. practice IPv6 first
2. practice DSCP marking with `ping -Q`
3. practice host-side `tc prio`
4. practice host-side RED/ECN
5. only then try leaf access-port `tc`
6. treat PFC as conceptual only in this virtual build
7. finish with `ib\_write\_bw`

18. Final Workbook Summary

This workbook lets you practice additional QoS and IPv6 concepts on top of the already working lab without replacing the known-good base fabric.

The strongest practical outcomes in this topology are:

- end-to-end IPv6 over the VXLAN bridge domain
- DSCP marking and verification

- Linux software queueing for priority and RED/ECN
- end-to-end RDMA regression testing

The main limitation remains:

- true PFC and hardware congestion-management behavior are not fully reproducible in this EVE-only environment

That is a platform boundary, not a design gap in the workbook.